

HDOC Day-32

Matrix Addition and Multiplication Using Variable-Length 2D Arrays

Aim

To write a menu-driven C program using separate functions and variable-length 2D arrays to demonstrate that matrix addition is commutative and matrix multiplication is not commutative in general.

Algorithm

1. Start the program and display a menu: (1) Matrix Addition, (2) Matrix Multiplication, (3) Exit.
2. Accept the user choice and process it using a switch statement.
3. For matrix addition, input common rows and columns, read two variable-length matrices A and B, calculate $A + B$ and $B + A$ using separate functions, display both results, and compare corresponding elements to demonstrate commutativity.
4. For matrix multiplication, input dimensions of A and B such that $A \times B$ is possible. To also calculate $B \times A$ for comparison, use square matrices of the same order in the test case. Calculate and display $A \times B$ and $B \times A$ using a separate multiplication function.
5. Compare $A \times B$ with $B \times A$ and display that multiplication is not commutative when the products differ.
6. Display an error message for an invalid menu choice and repeat the menu until Exit is selected.
7. Stop the program.

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c]);
void displayMatrix(int r, int c, int a[r][c]);
void addMatrices(int r, int c, int a[r][c], int b[r][c], int sum[r][c]);
void multiplyMatrices(int n, int a[n][n], int b[n][n], int product[n][n]);
int equalMatrices(int r, int c, int a[r][c], int b[r][c]);

int main(void)
{
    int choice;

    do
    {
        printf("\n----- MATRIX OPERATIONS MENU -----\n");
        printf("1. Prove Matrix Addition is Commutative\n");
        printf("2. Prove Matrix Multiplication is Not Commutative\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
```

```

case 1:
{
    int r, c;
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

    if (r <= 0 || c <= 0) {
        printf("Invalid dimensions!\n");
        break;
    }

    int A[r][c], B[r][c], AB[r][c], BA[r][c];
    printf("Enter Matrix A:\n");
    readMatrix(r, c, A);
    printf("Enter Matrix B:\n");
    readMatrix(r, c, B);

    addMatrices(r, c, A, B, AB);
    addMatrices(r, c, B, A, BA);

    printf("A + B:\n");
    displayMatrix(r, c, AB);
    printf("B + A:\n");
    displayMatrix(r, c, BA);

    if (equalMatrices(r, c, AB, BA))
        printf("A + B = B + A. Matrix addition is commutative.\n");
    break;
}

case 2:
{
    int n;
    printf("Enter order of the two square matrices: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid order!\n");
        break;
    }

    int A[n][n], B[n][n], AB[n][n], BA[n][n];
    printf("Enter Matrix A:\n");
    readMatrix(n, n, A);
    printf("Enter Matrix B:\n");
    readMatrix(n, n, B);

    multiplyMatrices(n, A, B, AB);
    multiplyMatrices(n, B, A, BA);

    printf("A x B:\n");
    displayMatrix(n, n, AB);
    printf("B x A:\n");
    displayMatrix(n, n, BA);

    if (!equalMatrices(n, n, AB, BA))
        printf("A x B != B x A. Matrix multiplication is not commutative.\n");
    else
        printf("For these matrices A x B = B x A; this special case does not make
multiplication commutative in general.\n");
    break;
}

```

```

    }

    case 3:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice! Please enter 1 to 3.\n");
    }
} while (choice != 3);

return 0;
}

void readMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void displayMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void addMatrices(int r, int c, int a[r][c], int b[r][c], int sum[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];
}

void multiplyMatrices(int n, int a[n][n], int b[n][n], int product[n][n])
{
    int i, j, k;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            product[i][j] = 0;
            for (k = 0; k < n; k++)
                product[i][j] += a[i][k] * b[k][j];
        }
}

int equalMatrices(int r, int c, int a[r][c], int b[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            if (a[i][j] != b[i][j])
                return 0;

    return 1;
}

```

Test Case Table

Test Case	Operation	Input	Expected Result
1	Addition	$A = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}$, $B = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}$	$A+B = B+A = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$
2	Addition	$A = \begin{bmatrix} 2 & 0 \\ 4 & 5 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 3 \\ 6 & 7 \end{bmatrix}$	Both sums are $\begin{bmatrix} 6 & 5 \\ 7 & 10 \end{bmatrix}$
3	Multiplication	$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 3 & 4 \\ 1 & 0 \end{bmatrix}$	$A \times B = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$; $B \times A = \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}$; not equal
4	Multiplication	$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$, $B = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$	$A \times B$ differs from $B \times A$
5	Invalid Menu	Choice = 4	Invalid choice message
6	Exit	Choice = 3	Program exits

Result

The menu-driven C program was successfully implemented using separate functions and variable-length 2D arrays. The addition results demonstrate $A + B = B + A$, while suitable multiplication test matrices demonstrate $A \times B \neq B \times A$, showing that matrix multiplication is not commutative in general.